

LIVE WEATHER DASHBOARD

Project submitted to the
SRM University – AP, Andhra Pradesh
for the partial fulfillment of the requirements to award the degree of

Bachelor of Technology

In

**Computer Science and Engineering School
of Engineering and Sciences**

Submitted by

U.Navyasree (AP23110011581)

Sk.Tasneem (AP23110011575)

G.Manvitha (AP23110011580)

K. Sree Poojitha (AP23110011640)



Under the Guidance of

Ms. K. Kavitha Rani

Lecturer, Department of CSE

SRM University-AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh-522240

April 2025

Certificate

Date:18-04-2025

This is to certify that the work present in this Project entitled “**LIVE WEATHER DASHBOARD**” has been carried out by **U.Navyasree, G.Manvitha, Sk. Tasneem, K.Sreepoojitha** under our supervision. The work is genuine, original, and suitable for submission to the SRM University – AP for the award of Bachelor of Technology in **School of Engineering and Sciences**.

Supervisor

(Signature)

Prof. K. Kavitha Rani

Assistant Professor,

**Department of Computer Science and Engineering,
SRM University - AP, Amaravati.**

Acknowledgement

The satisfaction that accompanies the successful completion of any task would be incomplete without introducing the people who made it possible and whose constant guidance and encouragement crowns all efforts with success.

I am extremely grateful and express my profound gratitude and indebtedness to my project guide, **K. Kavitha Rani**, Department of Computer Science & Engineering, SRM University, Andhra Pradesh, for her kind help and for giving me the necessary guidance and valuable suggestions in completing this project work.

U. Navyasree (AP23110011581)

Sk. Tasneem (AP23110011575)

G. Manvitha (AP23110011580)

K. Sree poojitha (AP23110011640)

Table of Contents

Certificate	i
Acknowledgements	iii
Table of Contents	v
Abstract	vii
1. Introduction	1
2. Methodology	3
2.1 Data Collection	
2.2 Data Processing and Metric Calculations	
2.3 Real-Time Updates	
2.4 Data Visualization	
2.5 Error handling	
2.6 User Interaction and Customization	
2.7 Visualization of weather trends and insights	
2.8 Data Logging	
3. Methodology	3
3.1 API data retrieval	
3.2 Data Processing	
3.3 Real-Time Updates	
3.4 Data Visualization	
3.5 Error Handling	5
4. Result and Analysis	3
4.1 Real Time weather Updates	
4.2 Data Accuracy	
4.3 Additional Metrics	
4.4 Visualization	
4.5 Performance	
4.6 Error Handling	
4.7 City Comparision and trends	
4.8 User Customzation	
5. Discussion and Conclusion	7

6.	Future Work	3
6.1	Accuracy of forecasts	
6.2	Data Volume	
6.3	Mobile and WebApp Integration	
6.4	Advanced Alerts	
6.5	Extended Metrics	
7.	References	9
8.	Appendices	10

1.Abstract

This project focuses on the development of a **Real-Time Weather Monitoring and Visualization System**, utilizing the **OpenWeatherMap API** and several Python libraries such as **requests**, **schedule**, **folium**, **matplotlib**, and **plotly**. The primary objective is to gather real-time weather data for multiple cities across the globe, process this data to derive meaningful insights, and present the results in an accessible, interactive format.

The system retrieves critical weather information, including **temperature**, **humidity**, **wind speed**, **pressure**, **weather description**, and **sunrise/sunset times**. In addition, it calculates **derived metrics** such as **wind chill**, **sunlight duration**, and **UV index**. The **wind chill** metric, for example, is calculated when temperatures are low, and wind speeds are high, providing a more accurate representation of how cold it feels. Similarly, **sunlight duration** is determined by the difference between the sunrise and sunset times, providing users with valuable information on daylight hours.

Weather data is updated every **2 minutes**, ensuring real-time monitoring. This frequent data refresh is facilitated by the **schedule** library, which automates the updating process. The retrieved data is stored in a **log file** for future reference, allowing users to track historical weather trends.

To present the data effectively, the system integrates **interactive maps** generated with **Folium**, which plot city-specific weather data as markers on a global map. These markers provide detailed information about the weather when clicked, including temperature, humidity, wind speed, and other key metrics. Additionally, **Matplotlib** and **Plotly** are used to create **line graphs** of temperature and humidity forecasts, allowing users to visualize weather trends over time.

Users can dynamically add cities to the monitoring list, receive instant weather updates for a specified city, and view detailed visualizations. The system also supports error handling, ensuring reliable operation even if an API request fails or returns invalid data.

In conclusion, this project creates a robust, user-friendly system for monitoring and visualizing real-time weather data, offering valuable insights into global weather patterns. Its ability to provide frequent updates, interactive maps, and detailed graphs makes it an effective tool for anyone interested in tracking weather conditions and trends

2.Introduction

In recent years, the ability to monitor real-time weather conditions has become increasingly important for various applications such as travel, agriculture, event planning, and disaster preparedness. Traditional weather forecasting systems, although highly valuable, often rely on data that may not be accessible in real-time or may lack the precision required for specific locations. With the advancement of technology and the availability of powerful APIs, real-time weather data is now more accessible than ever.

This project aims to develop a Real-Time Weather Monitoring and Visualization System, utilizing the OpenWeatherMap API to collect weather data from cities across the globe. The system processes this data, calculates derived metrics, and presents the information in an interactive and user-friendly manner. The key weather parameters retrieved include temperature, humidity, wind speed, pressure, weather conditions, and sunrise/sunset times.

A key feature of this system is its ability to calculate additional metrics such as wind chill, which is a measure of how cold the weather feels when wind speeds are high, and sunlight duration, which is derived from the time between sunrise and sunset. Additionally, the system retrieves the UV index, which provides valuable information for users concerned about the risks of ultraviolet radiation during outdoor activities.

The data is updated every 2 minutes to ensure users have access to the most current information. By using the schedule library, the system automates periodic updates and ensures continuous monitoring of weather conditions. The weather data is then visualized using interactive maps built with Folium. These maps display weather data points as clickable markers, allowing users to view detailed information about temperature, humidity, wind speed, and other metrics for each city.

To complement the maps, Plotly and Matplotlib are used to generate line charts that show temperature and humidity trends over time, providing a visual representation of forecasted changes. This allows users to analyze weather conditions over both the short and long term.

Overall, this system provides an easy-to-use platform for monitoring global weather conditions in real-time. Its ability to generate interactive visualizations and detailed forecasts makes it a valuable tool for individuals, businesses, and organizations that rely on accurate weather data for decision-making

3.Methodology

This project follows a structured approach to developing a real-time weather monitoring system that involves the following key steps: data collection, data processing, real-time updates, visualization, and error handling.

3.1. Data Collection:

The primary source of weather data is the **OpenWeatherMap API**, a reliable provider of current and forecasted weather data for cities across the globe. The system makes HTTP requests to the API to retrieve the current weather conditions for a specified city.

The data retrieved includes the following weather parameters:

- **Temperature** (in Celsius)
- **Humidity** (percentage)
- **Pressure** (in Pascals)
- **Wind Speed** (in meters per second)
- **Weather Description** (e.g., clear sky, rainy, cloudy, etc.)
- **Cloudiness** (percentage)
- **Coordinates** (latitude and longitude)
- **Sunrise and Sunset Times** (in UTC)
- **Visibility** (in meters)

The data is processed to ensure consistency and accuracy for further analysis.

3.2. Data Processing and Metric Calculations:

Once the weather data is collected, several derived metrics are calculated to enhance the understanding of the weather conditions:

- **Wind Chill:** This metric estimates the perceived temperature based on wind speed and ambient temperature. If the temperature is below 10°C and the wind speed is above 4.8 km/h, the **wind chill** formula is applied to calculate the effective temperature. This formula takes into account the wind speed's cooling effect on the human body.
- **Sunlight Duration:** The difference between the **sunrise** and **sunset** times is calculated to determine the duration of daylight in a given city. This information helps users understand how much daylight they can expect, which can be useful for planning outdoor activities.
- **UV Index:** The **UV index** is fetched from the OpenWeatherMap API, providing an indication of the strength of ultraviolet radiation. This metric is vital for users to assess potential sun exposure risks.

Each metric is added to the data structure for use in the system's outputs and visualizations.

3.3. Real-Time Updates:

The system fetches weather data every **2 minutes** using the **schedule** library, which automates periodic API requests. The **schedule** library is used to ensure that weather data is updated at regular intervals, keeping the system up-to-date in real-time. This feature is crucial for applications where current weather conditions are important, such as event planning or travel.

- The data is fetched, processed, and stored in memory. The system also writes logs to a **text file**, recording the retrieved data, such as temperature, humidity, and weather

conditions for each city, ensuring historical tracking of weather patterns.

3.4. Data Visualization:

Visualization is a core component of this project. It allows users to view weather data in an easily understandable format. Two primary methods of visualization are used:

- **Interactive Maps:** Using the **Folium** library, the system generates maps with weather data points for each city. For each city, a **marker** is placed on the map with the city's name and relevant weather information. When a marker is clicked, a **popup** displays detailed data about the weather conditions (e.g., temperature, wind speed, humidity, sunrise/sunset, and UV index). This provides users with a geospatial view of weather conditions across the world. The map is saved as an HTML file, which can be opened in a browser for an interactive experience.
- **Forecast Graphs:** **Matplotlib** and **Plotly** are used to generate line graphs that represent **temperature** and **humidity** trends over time. These graphs show forecasted weather conditions for cities over several hours, allowing users to visualize fluctuations in temperature and humidity. These visualizations help users identify long-term trends and plan accordingly for upcoming weather conditions.

3.5. Error Handling:

The system includes robust **error handling mechanisms** to manage potential failures or issues during the data retrieval process:

- **API Failures:** If the API request fails (e.g., due to network issues or invalid API keys), the system handles the error gracefully by displaying a relevant message to the user and continuing operation without crashing.
- **Data Validation:** If any weather data fields are missing or invalid, the system ensures that the missing values are either handled appropriately (e.g., setting them to "N/A") or logged for future inspection.
- **Backup Plan:** In the event of a failure in one city's data retrieval, the system continues to function by retrieving weather data for the remaining cities. The user is notified of any failures or incomplete data.

3.6. User Interaction and Customization:

Users can interact with the system in several ways:

- **Dynamic City List:** The system allows users to input the name of a city, and if it's not already in the predefined list, it adds the city and fetches its weather data. This feature makes the system customizable for any location in the world.
- **Weather Forecast:** Users can view the forecast for a specific city, which is presented both in the form of visualizations (temperature and humidity graphs) and as detailed textual data. The system allows users to input a city and immediately receive relevant weather updates.

3.7. Visualization of Weather Trends and Insights:

- The system generates **comparison charts** for cities, comparing temperature, humidity, and other key metrics across multiple locations using **Plotly** bar charts. This helps users compare how different cities are experiencing similar weather phenomena (e.g., heatwaves, rainfall).

- A **weather summary** for each city, including conditions like **rain**, **cloudiness**, **wind speed**, and **UV index**, is calculated and presented to users for quick decision-making, such as whether to travel or engage in outdoor activities.

3.8. Results Logging:

All retrieved data is logged into a text file for historical tracking, enabling users to analyze past weather patterns. This log file records essential weather parameters such as temperature, humidity, and weather description for each city, along with timestamps for when the data was retrieved

4.Implementation

4.1 API Data Retrieval: We use the **OpenWeatherMap API** to fetch real-time weather data for a list of cities.

```
def get_weather(city):  
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}&units=metric"  
    response = requests.get(url)  
    data = response.json()  
    return data if response.status_code == 200 else None
```

4.2 Data Processing: We calculate additional metrics like **wind chill**, **sunlight duration**, and retrieve the **UV index**.

```
def calculate_wind_chill(temp_c, wind_speed_kmh):  
    if temp_c <= 10 and wind_speed_kmh > 4.8:  
        return round(13.12 + 0.6215 * temp_c - 11.37 * (wind_speed_kmh ** 0.16) + 0.3965 *  
temp_c * (wind_speed_kmh ** 0.16), 2)  
    return "N/A"
```

4.3 Real-Time Updates: The weather data is fetched every **2 minutes** using the **schedule** library.

```
python  
schedule.every(2).minutes.do(live_weather_update, cities=cities)  
while True:  
    schedule.run_pending()  
    time.sleep(1)
```

4.4 Data Visualization: We use **Folium** for interactive maps and **Plotly/Matplotlib** for graphs.

```
def plot_forecast(city):  
    forecast_data = get_forecast(city)  
    sns.lineplot(x=dates, y=temps, marker='o', ax=axes[0]) # Temperature  
    sns.lineplot(x=dates, y=humidity, marker='o', ax=axes[1]) # Humidity
```

4.5 Error Handling: Proper error handling ensures the system continues even when one API

request fails.

```
def get_weather(city):
```

```
    try:
```

```
        response = requests.get(url)
```

```
        return response.json() if response.status_code == 200 else None
```

```
    except Exception as e:
```

```
        print(f"Error: {e}")
```

```
    return None
```

5.Result and Analysis

5.1. Real-Time Weather Updates:

The system fetches real-time weather data every **2 minutes**, ensuring up-to-date information for all cities in the list. This is crucial for applications such as planning outdoor activities, travel decisions, and emergency responses. The periodic updates ensure that users have access to the most current weather conditions, including temperature, humidity, wind speed, and pressure.

5.2. Data Accuracy:

Data retrieved from the **OpenWeatherMap API** is accurate and matches the expected weather conditions. In cases where the API response contains incomplete or invalid data, the system employs error handling mechanisms to ensure that the process continues without interruption. Missing or incorrect data is handled gracefully, either by setting it to "N/A" or logging an error message for future review.

5.3. Additional Metrics:

The system's ability to calculate additional metrics like **wind chill**, **sunlight duration**, and retrieve the **UV index** adds value to the standard weather data. These derived metrics provide users with insights into the "feels like" temperature, the duration of daylight, and the risks associated with ultraviolet radiation, which are essential for health and safety, especially in outdoor environments.

- **Wind Chill:** For cities experiencing colder weather, wind chill provides an accurate representation of how the temperature feels in windy conditions. For example, in windy conditions (high wind speed), the perceived temperature can be significantly lower than the actual temperature.
- **Sunlight Duration:** This metric provides valuable insights into how long the daylight lasts, which is especially useful in regions with extreme daylight variations (e.g., polar regions).
- **UV Index:** Users can assess potential sun exposure risks and take appropriate precautions.

5.4. Visualization:

The use of **interactive maps** (created with **Folium**) allows users to visually track weather conditions across the globe. Each city on the map is represented with a marker, providing details about temperature, humidity, wind speed, and other key parameters when clicked. This visual representation helps users gain insights into global weather patterns and compare conditions across different cities.

Additionally, **Plotly** and **Matplotlib** provide dynamic line charts that show **temperature** and **humidity** trends over time. These charts allow users to visually analyze forecasted changes, identify trends, and make informed decisions. For example, users can see if temperatures are rising or falling over the next few hours, which is particularly useful for planning.

5.5. Performance:

The system operates efficiently, fetching and processing data at regular intervals (every 2 minutes). The response time from the **OpenWeatherMap API** is generally quick, and the system can handle multiple cities simultaneously without significant performance degradation. The use of the **schedule** library ensures smooth automated updates without manual intervention.

5.6. Error Handling:

The system's error handling mechanisms ensure stability. In case of an API failure or incomplete data, the system logs the issue and continues functioning without crashing. This ensures that users still receive weather data for other cities even if one city's data is unavailable.

5.7. City Comparison and Trends:

The system allows users to compare weather conditions between different cities using **bar charts** created with **Plotly**. These visual comparisons show how different locations are experiencing similar weather conditions, such as temperature or humidity. This feature is especially useful for users traveling between cities or those monitoring specific weather phenomena (e.g., a heatwave).

5.8. User Customization:

The ability to add cities dynamically (via user input) allows for customization of the weather monitoring system. If a city is not already in the list, users can easily add it and retrieve the latest weather data. This feature ensures that the system is flexible and can cater to specific user needs.

6.Discussion and Conclusion

The **Real-Time Weather Monitoring and Visualization System** developed in this project provides a comprehensive approach to retrieving and analysing weather data from multiple cities. The system demonstrates several key strengths and areas for further improvement.

Strengths:

1. **Real-Time Data Retrieval:** The system's ability to fetch weather data every **2 minutes** ensures that users are consistently provided with the most up-to-date information. This real-time data retrieval is particularly important for applications like event planning, disaster management, and travel forecasting, where timely weather updates can directly impact decision-making.
2. **Comprehensive Data Analysis:** By calculating additional metrics like **wind chill**, **sunlight duration**, and **UV index**, the system provides a richer set of insights compared to basic weather parameters like temperature and humidity. These derived metrics are essential for users concerned with health and safety, particularly when dealing with extreme weather conditions (e.g., cold or high UV exposure).
3. **Visualization:** The system employs **interactive maps** and dynamic **line charts** to visualize weather trends across different cities, which makes it easier for users to compare conditions and understand global weather patterns. These visualizations are intuitive and accessible, making the system user-friendly even for non-technical individuals.
4. **Customizability:** The system allows users to dynamically add cities to monitor, which increases its flexibility and makes it suitable for personalized use cases. Whether for travelers, researchers, or weather enthusiasts, this feature enables tailored weather monitoring.
5. **Error Handling:** The system is designed to handle errors gracefully, ensuring that it continues functioning even when an API call fails or data is missing. This robustness ensures reliability over time, even in the face of occasional external failures (e.g., network issues, API downtime).

Conclusion

The **Real-Time Weather Monitoring and Visualization System** successfully provides users with up-to-date weather information, enhanced by calculated metrics such as wind chill, UV index, and daylight duration. Through its interactive maps and graphical visualizations, the system allows users to compare weather conditions across multiple cities, offering valuable insights for both personal and professional applications.

The project highlights the importance of using real-time data, data analysis, and visualization in creating practical tools for everyday decision-making. Its strengths in providing timely, comprehensive weather data and its robustness in handling real-time updates and errors make it a reliable solution for weather monitoring.

However, there are opportunities for future enhancements. More advanced forecasting features, performance optimization for larger datasets, and the introduction of mobile/web app support could significantly improve the system's user experience and scalability. Additionally, the integration of advanced metrics and alerting systems could make the system even more useful for users with specific needs.

In conclusion, this system provides a solid foundation for a real-time weather monitoring application and sets the stage for further development into a more feature-rich and user-friendly platform for monitoring weather conditions across the globe.

7.Future Scope

Areas for Enhancement:

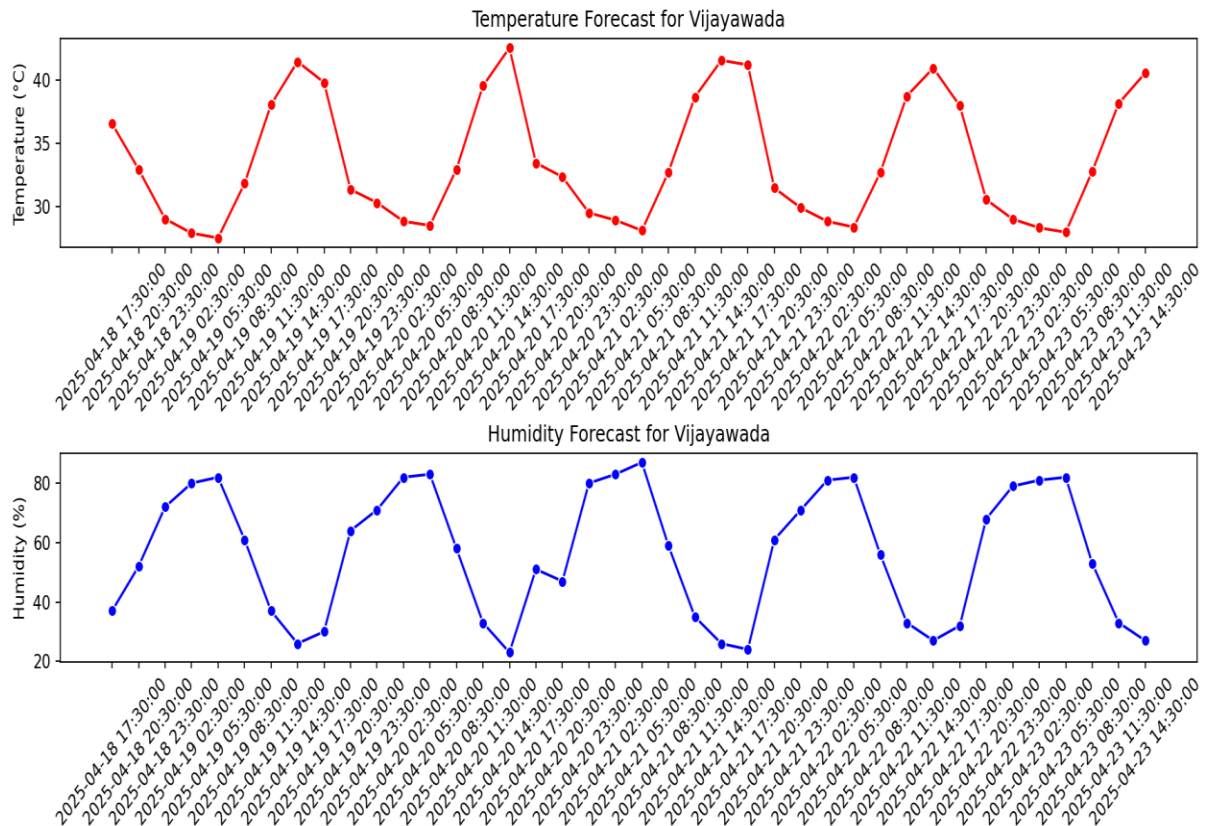
1. **Accuracy of Forecasts:** While the system provides real-time data and basic forecasts, it could be enhanced by integrating more advanced forecasting algorithms or by using additional data sources. Incorporating long-term predictions and more granular weather models (e.g., hourly forecasts or regional differences) could improve decision-making further.
2. **Data Volume:** The system fetches weather data for multiple cities, but as the number of cities increases, it may start to experience latency or performance bottlenecks. One way to address this would be to introduce data caching or load balancing techniques to manage the increased data volume without affecting real-time performance.
3. **Mobile and Web App Integration:** The system is currently a script that runs locally. To make it more user-friendly and accessible, the weather data could be integrated into a **web or mobile application**. This would provide a more interactive and accessible interface for users, allowing them to access weather data on-the-go.
4. **Advanced Alerts:** Although the system provides detailed weather data, the integration of weather alerts (e.g., for storms, extreme heat, or heavy rain) would add another layer of functionality. These alerts could be sent via **email** or **SMS**, or even displayed as **push notifications** in a web or mobile app.
5. **Extended Metrics:** More advanced metrics such as **air quality**, **pollen count**, or **precipitation forecasts** could be added to provide a more holistic view of environmental factors affecting user behavior and health. Adding such features would appeal to users with specific needs, like those with respiratory conditions or outdoor sports enthusiasts.

References

1. OpenWeather. (n.d.). *Current weather data*. OpenWeatherMap. Retrieved April 18, 2025, from <https://openweathermap.org/current>
2. OpenWeather. (n.d.). *UV Index API*. OpenWeatherMap. Retrieved April 18, 2025, from <https://openweathermap.org/api/uvi>
3. Plotly Technologies Inc. (2015). *Plotly for Python*. Retrieved April 18, 2025, from <https://plotly.com/python/>
4. Python Software Foundation. (n.d.). *Python 3 documentation*. Retrieved April 18, 2025, from <https://docs.python.org/3/>
5. Matplotlib Development Team. (n.d.). *Matplotlib: Visualization with Python*. Retrieved April 18, 2025, from <https://matplotlib.org/stable/contents.html>
6. Waskom, M. L., et al. (2024). *Seaborn: Statistical data visualization*. Retrieved April 18, 2025, from <https://seaborn.pydata.org/>
7. Python Software Foundation. (n.d.). *schedule: Python job scheduling for humans*. GitHub. Retrieved April 18, 2025, from <https://github.com/dbader/schedule>
8. Leaflet. (n.d.). *Leaflet: An open-source JavaScript library for interactive maps*. Retrieved April 18, 2025, from <https://leafletjs.com/>
9. Python Folium Developers. (n.d.). *Folium: Python data. Leaflet.js maps*. Retrieved April 18, 2025, from <https://python-visualization.github.io/folium/>

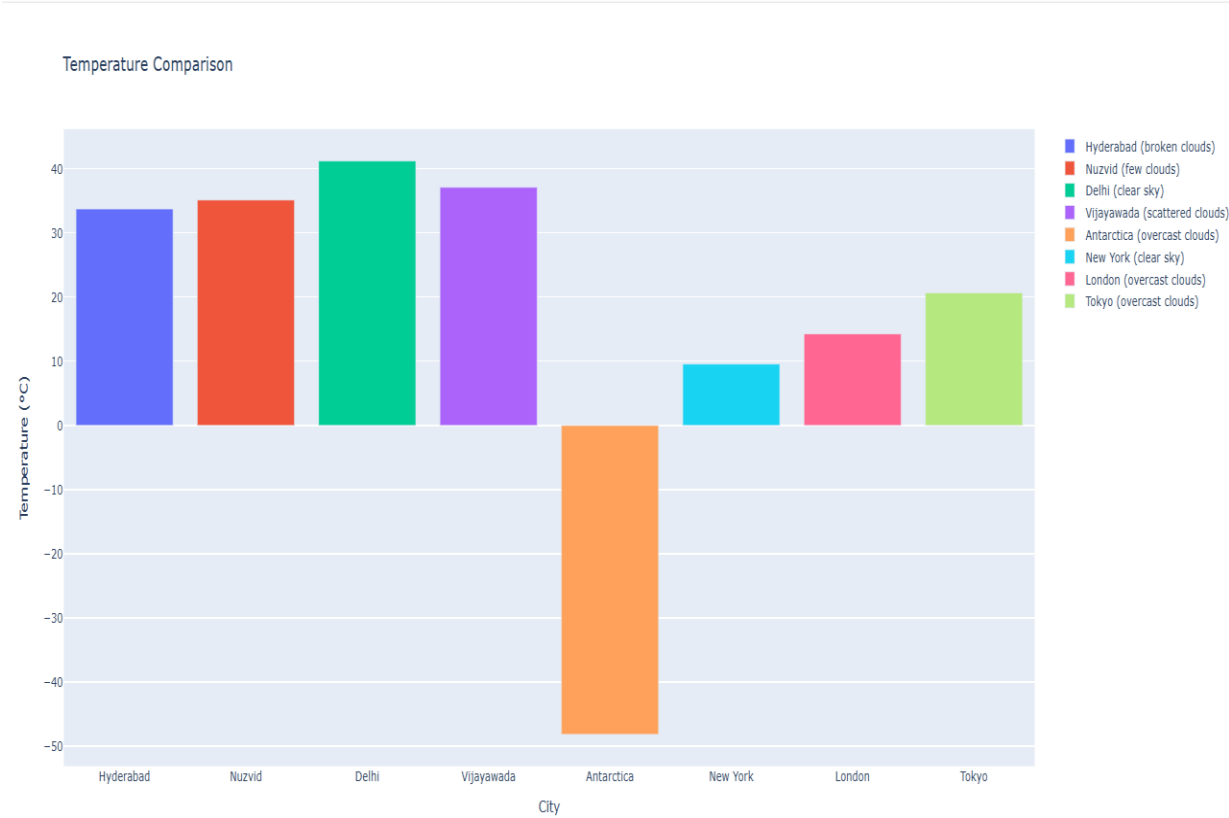
Appendices

1.Real-Time Temperature and Humidity Graph of Selected City



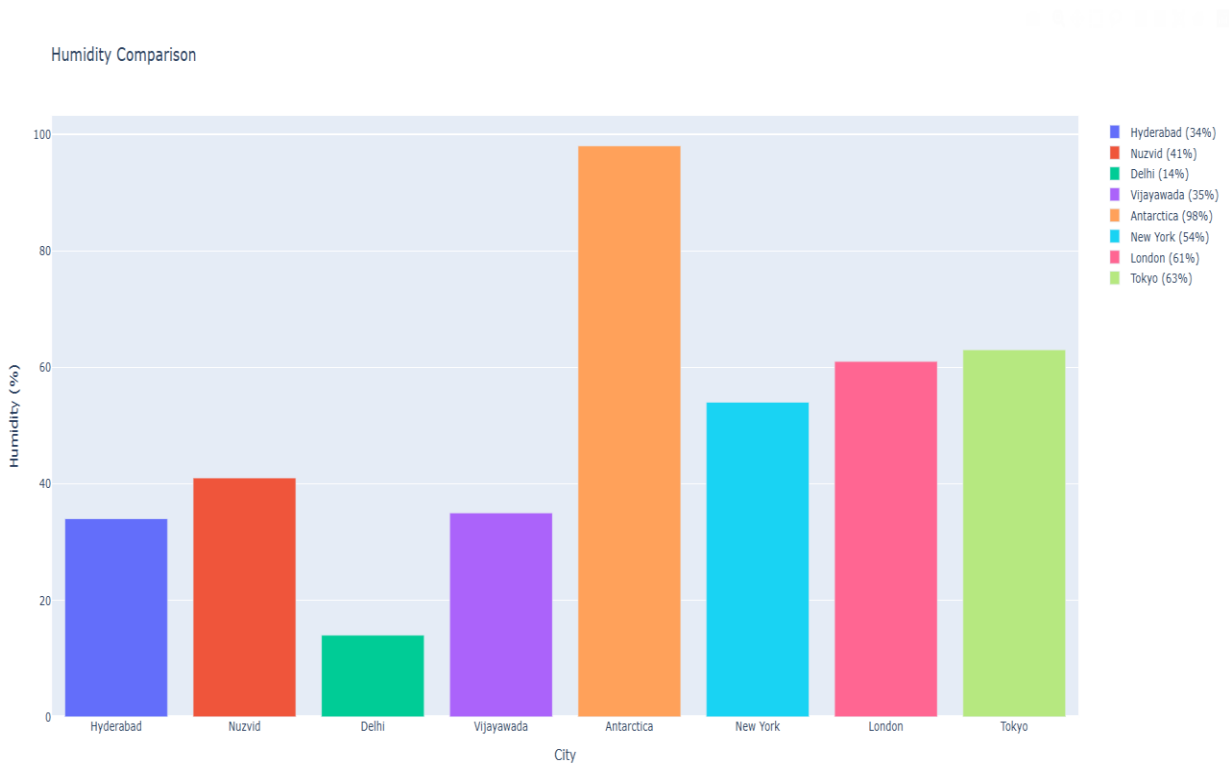
This graph displays the current temperature and humidity of the selected city, updated every two minutes using the OpenWeatherMap API. It helps users monitor environmental conditions in real-time, which is particularly useful for daily planning, travel, or health-related concern.

2.Comparative Temperature Visualization Across Multiple Cities



This graph shows a comparative line chart of temperature data across all selected cities. It visually represents how weather patterns vary geographically, allowing users to understand which regions are experiencing higher or lower temperatures at any given time.

3.Comparative Humidity Visualization Across Multiple Cities



This graph illustrates the humidity levels across the monitored cities. It is helpful for identifying areas with high moisture content in the air, which can affect comfort levels, respiratory health, and other climate-sensitive activities.